Hantro VC9000D

Video Decoder Wrapper Layer

API User Manual

Document Revision 1.03

28 March 2024

This document is compatible with the following product:

Hantro VC9000D software release version 2.4.76

VERISILICON

LEVEL A: PROPRIETARY INFORMATION

Legal Notices

Preface

This chapter introduces the *Hantro VC9000D Video Decoder Wrapper Layer API User Manual*.

About This Document

This document introduces the application programming interface (API) of the decoder wrapper later (DWL) of Hantro VC9000D multi-format video decoder IP. It describes the data types, enumerations, structures, and functions in the API, which is written in C code. It also provides a programming overview and examples for your reference.

Additional Reading

- Hantro VC9000D Video Decoder Hardware Features
- Hantro VC9000D Video Decoder Memory Buffer and Format Organization

Table of Contents

1 Overview

This document describes the Application Programming Interface (API) of the decoder wrapper layer (DWL) for VC9000D video decoder hardware.

The multi-format VC9000D video decoder can be configured as multi-core to support up to 8192x4096 decoding. It features high quality, high throughput, low power consumption, and negligible load on the host CPU.

The functionality of the VC9000D video decoder is exposed to an application through a decoder API on top of all the codec algorithms and the decoder software. The decoder software is built on an abstraction layer called DWL. The DWL includes the operating system abstarction and the hardware abstraction, and encapsulates all the required system-dependent resources. This structure eliminates the need of modifying the decoder software when porting the decoder software to a system.

This document describes the data types, enumerations, structures, and functions in the VC9000D DWL API, which is written in ANSI C. It also provides a guide on how to integrate the VC9000D video decoder software to your system to your system by using the DWL API.

2 Software Integration

The DWL interface describes the service control software requirements of the environment. You can access and implement system-specific functionalities through this interface. The software delivery package provides an implementation example on Linux.

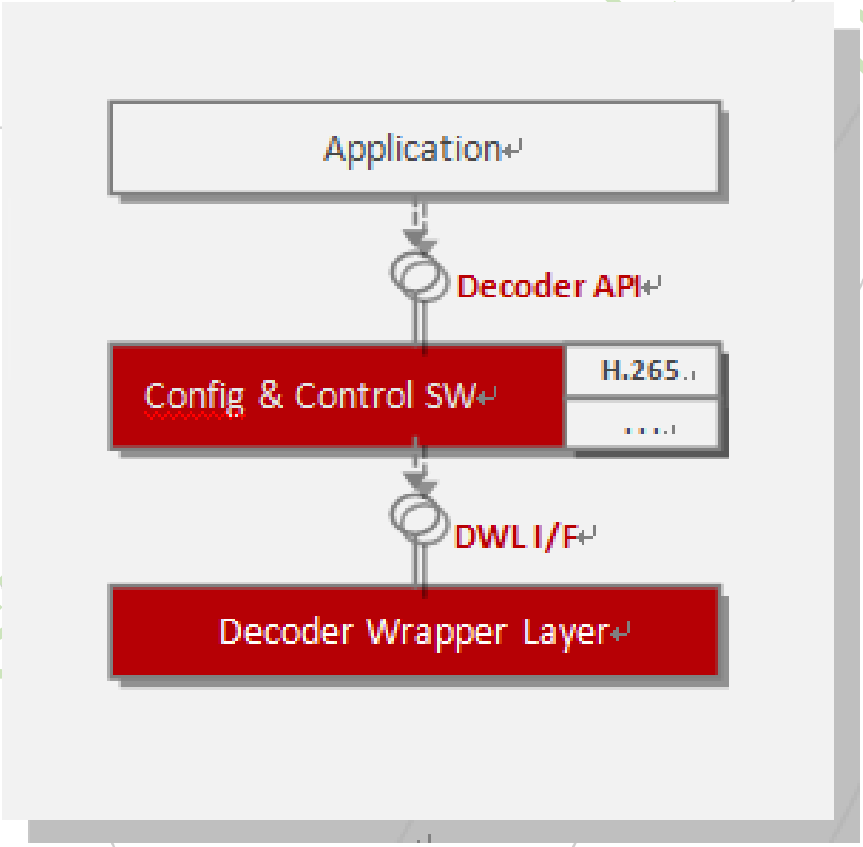

Figure 2.1 Decoder Software Structure

2.1 Control Software Configuration

When compiling the control software, you must update some global decoder information to match the system specification. This information is provided in the *deccfg.h* file and you may need to

update it for proper configuration.

2.2 Register Access

The decoder hardware is configured by using a set of memory-mapped I/O registers. You must map these registers to the address space of the control software and provide read and write access to these registers through `DWLReadReg()` and `DWLWriteReg()`.
When hardware configuration is completed, the `DWLEnableHw()` function enables the hardware with a final writing to one specific register. This specific hardware starting point allows flexible DWL implementation, in which the register accesses can be buffered. Once buffered, the cached values are written to the hardware when `DWLEnableHw()` is called and refreshed by reading back from the registers. This refresh is alway executed after a hardware status change, which is signaled by a return from DWLWaitHwReady().
The software can force the hardware to stop by calling `DWLDisableHw()`. This function disables the hardware by writing to a specific register, whose access should not be buffered.

2.3 Power and Clock Control

When decoding one frame, the power and clock of the decoder must be active between calling `DWLEnableHw()` and the moment when the hardware finishes processing the frame. The hardware will not stop processing until `DWLWaitHwReady()` is returned to unblock the control software. When the power and clock is enabled, the software can obtain register values using `DWLReadReg()` and `DWLWriteReg()`. If the register values are used outside the power and clock active period, you must read these values out and stores them into a cached memory area called **dwl_shadow_regs**.

2.4 Memory

The accelerator uses the following types of memory buffers allocated through DWL.

- Software-only buffers
  This type of buffers can only be accessed by the software. They are typically allocated using `malloc()` and `calloc()`, which are the standard C memory allocation routines. The DWL provides a series of functions, such as `DWLmalloc()`, `DWLcalloc()`, `DWLmemcpy()`, and `DWLfree()`, to process the software-only memory resources.

- Hardware-only buffers
  This type of buffers can only be accessed by the hardware. They are linear and physically contiguous. The DWL provides `DWLMallocLinear()` and `DWLFreeLinear()` to manage such buffers.

- Software and hardware shared control data buffers
  This type of buffers share the algorithmic control data between the software and the hardware. They are linear and physically contiguous. The DWL provides `DWLMallocLinear()`, `DWLFreeLinear()`, and `DWLDMATransData()` to manage such buffers.
  For more information about the decoder memory buffers, refer to *Hantro VC9000D Memory Buffer and Format Organization*.

2.5 Hardware and Software Synchronization

The software and hardware components need to synchronize their operations. When the software expects the hardware to finish its current task, it calls `DWLWaitHwReady()`, which must be blocked until the hardware completes processing.

2.5.1 Interrupt and Polling

The hardware can use the following modes to indicate that it has completed processing. The integrator decides which mode to use.

- Interrupt mode In this mode, an interrupt service routine (ISR) must be established. Only after the ISR is triggered by an interrupt from the underlying hardware, VCCORE, can the sifware be unblocked. The specific implementation varies by target operating system and hardware platform.

- Polling mode In this mode, the DWL implementation reads the relevant hardware status registers repeatedly in an infinite loop to detect when the hardware is ready and exits the loop when the hardware completes processing.
  To avoid infinite deadlocks, you can use both software timeout, which is system-specific, and hardware timeout, which is enabled or disabled by the interrupt bit, bit 18, in the decoder device configuration register, swreg[1]. In the case of decoder deadlock, you need to wait for `DWLWaitHwReady()` to return `DWL_HW_WAIT_TIMEOUT` before implementing timeout.

2.6 Multi-Instance Decoding

The Hantro video decoder is designed as context-free to process one picture frame. After decoding one stream, the hardware resets its internal state. All contexts used to decode one stream are obtained from registers. Such a design makes the decoder capable of decoding multiple streams in a frame-based time-division way.

In the software, each stream is maintained as one instance. You can create multiple software instances to drive one single hardware block on a time-sharing basis. For multi-instance support, the software needs to provide a mechanism to make each instance access the hardware exclusively. Such a mechanism should be implemented by `DWLReserveHw()` and `DWLReleaseHw()`. These two functions grant and release exclusive hardware access to a given software instance.

In the implementation of `DWLReserveHw()`, the instance queries for hardware access. If the hardware is occupied by other instances, the instance waits. When the hardware is available, this function becomes the owner of the hardware, and the corresponding instane can access the hardware exclusively. After the owner instance finishes the hardware-related operation, `DWLReleaseHw()` is called to release the hardware access.

`DWLReserveHw()` and `DWLReleaseHw()` are called by the control-software function `VCDecDecode()`. The reference Linux implementation is provided. The exclusive access is maintained in the Linux kernel driver to make the multi-instance work in both a multi-thread and multi-process manner.

The application can use the following methods to manage multiple streams:

- The application maintains and manages multiple decoder instances.
  After one frame is decoded, the application can switch to another frame in either the same instance or a different instance.
- The application utilizes multiple processes, with each process managing one stream.
  The decoder driver incorporates hardware resource protection to avoid conflicts between different processes. After each frame is decoded, the hardware resource is released for other processes to use.

2.7 Threading

The control code functionality is split into two threads as illustrated in the following figure.

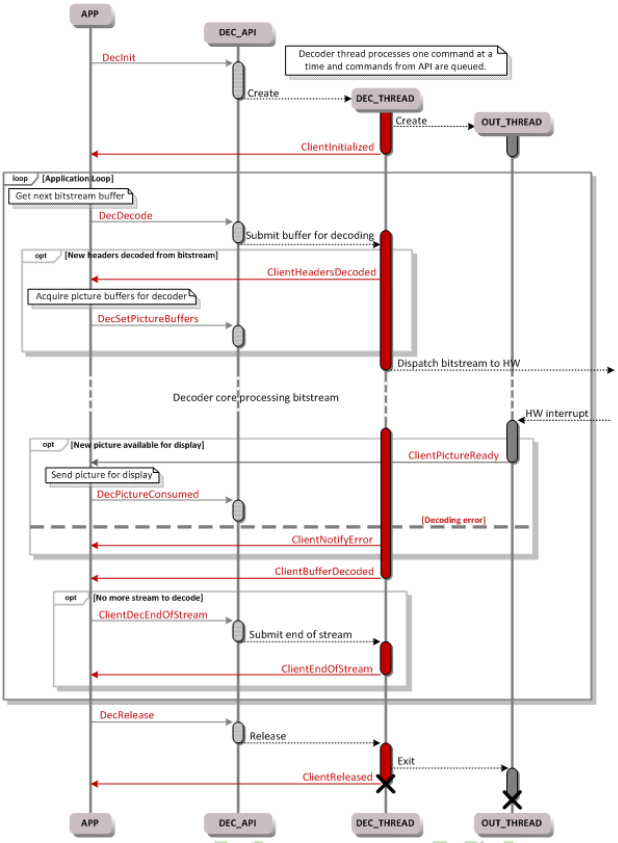

Figure 2.2 DWL Control Code Flow

Each command from the application is encapsulated into a message and passed to the decoder thread. Decoder threads pick up these messages in the order they arrive. The application thread may still block when the message queue fills up. It will be unblocked when the decoder thread finishes processing the current message and reads the next message from the queue. When messages that require hardware processing are encountered, the decoder thread prepares the hardware for the execution and enables the hardware accelerator.

When the hardware finishes decoding pictures, each correctly decoded picture is dispatched to the second thread, which pushes each picture back to the application in the display order through the application-provided callback function `ClientPictureReady()`. This threading structure ensures that the application thread can easily queue buffers for decoding on the accelerator and that the accelerator will not be blocked by the picture post-processing implemented by the application.

Threads are managed by using a subset of Portable Operating System Interface (POSIX) threading functionality that is defined in the DWL.

2.8 OS Porting Example

The decoder software is designed in a way that the DWL hides the OS limitations and OS-specific issues from the algorithms and decoder software. In most cases, the DWL is the only part that needs attention when porting, but exceptions might still exist. Below a Linux porting example is described. The decoder software is built up as a user space static library.

2.8.1 DWL Initialization and Release

The DWL initialization routine is provided so that the DWL can initialize itself before any other DWL call is made. The availability of the resources required by the decoder must be ensured at this initialization point. The resources needed by the decoder software are:

- Linear and contiguous physical memory for hardware-related buffers
- Memory needed by the software
- Hardware register access
- Software and hardware synchronization
- Hardware sharing

2.8.2 Linear Memory Management

One method for getting linear, contiguous physical memory in Linux is to instruct the kernel at loading time not to use the whole available RAM memory. Linux can leave a part of the RAM memory unused which an application can access by mapping it with the help of the memory device /**dev**/**mem**. The same Linux memory device can be used to map the hardware I/O registers to a virtual address space. In this way, the decoder can have direct pointer access to them. The allocations of all linear buffers are performed at the same phase of decoding, so as the releasing of these buffers. No individual linear buffer can be reallocated without allocating the others.
A linear memory management device example is provided in `software/linux/memalloc`. The memalloc device provides a method for sharing linear memory between multiple decoder instances.
The needed function provided in `dwl_linux.c` are as belows:

- `DWLFreeLinear()`
- `DWLMallocLinear()`

2.8.3 Software-Only Memory Handling

All the DWL calls related to the software-only memory can be implemented by using their ANSI C counterparts, as can be seen in the example implementation source files. If for any reason the full ANSI C library is not available in the target system, these functions have to be implemented in a more system specific way, but with offering the same functionality.
The needed function provided in `dwl_linux.c` are as belows:

- `DWLmalloc()`
- `DWLfree()`

`DWLmalloc()` and `DWLfree()` are implemented by using ANSI C counterparts `malloc()` and `free()`.

2.8.4 Hardware Register Access

The hardware registers are mapped to the user space during DWL initialization, when a pointer to the base of the register bank is provided.
The needed function provided in `dwl_linux.c` are as belows:

- `DWLReadReg()`
- `DWLWriteReg()`

`DWLReadReg()` writes and `DWLWriteReg()` reads a register according to a specified offset.

2.8.5 Software and hardware synchronization

The software and hardware synchronization can be accomplished either with hardware polling or interrupt (IRQ). The polling is done by reading the hardware status register. If IRQ is used, a kernel device driver has to be created because an IRQ can be served only in the Linux kernel space. To notify the decoder in user space, the kernel device driver can send a SIGIO signal to the decoder listener thread. This signal is sent after an IRQ is received from the decoder hardware.

The advantages of the polling method are that the whole decoder software can reside in the user space and does not need a kernel driver for handling IRQs. The drawback is that the polling is usually done at fixed time intervals. If this interval is short, the decoder will use more CPU, and when it is longer, the overall decoding performance is affected.

The IRQ-based method is slightly more complicated to implement but it guarantees a better decoding performance. However, the IRQ response latency and any context switch will cause performance reduction. The decoder software can sleep during the IRQ wait and thus, freeing the CPU for other tasks.
Two examples for the Linux DWL implementation are provided as example function called `DWLWaitHwReady()` in the file `dwl_linux_hw.c`.

2.8.6 Hardware Sharing

Global system semaphores can be used to implement an exclusive access granting mechanism for the hardware. In this way, multiple processes can use the same hardware. The wrapper layer instance can identify which type of client or decoder is requesting the access.

3 Module Documentation

3.1 COMMON

Marco Definition

- #define **DWL_MEM_TYPE_CPU** 0x0000U

  only CPU use the buffer. non-secure CMA memory.
- #define **DWL_MEM_TYPE_SLICE** 0x0001U

  VPU can read, CAAM can write.
- #define **DWL_MEM_TYPE_DPB** 0x0002U

  VPU can read and wirte, Render can read.
- #define **DWL_MEM_TYPE_VPU_WORKING** 0x0003U

  VPU can read, CPU can write. non-secure memory.
- #define **DWL_MEM_TYPE_VPU_WORKING_SPECIAL** 0x0004U

  VPU can read, CPU can read and wirte. only for VP9 counter context table.
- #define **DWL_MEM_TYPE_VPU_ONLY** 0x0005U

  only VPU can read and wirte the buffer.
- #define **DWL_MEM_TYPE_VPU_CPU** 0x0006U

  VPU&CPU both can read and write the buffer.
- #define **DWL_MEM_TYPE_DMA_DEVICE_ONLY** 0x0100U

  only host or only device use the buffer.
- #define **DWL_MEM_TYPE_DMA_HOST_TO_DEVICE** 0x0200U

  the buffer need transfer frome host to device.
- #define **DWL_MEM_TYPE_DMA_DEVICE_TO_HOST** 0x0300U

  the buffer need transfer frome device to host.
- #define **DWL_MEM_TYPE_DMA_HOST_AND_DEVICE** 0x0400U

  the buffer need transfer in both directions.

3.1.1 Enumeration Type

3.1.1.1 DWLDMADirection

enum DWLDMADirection

Enumerator

HOST_TO_DEVICE	copy the data from host memory to device memory
DEVICE_TO_HOST	copy the data from device memory to host memory

3.1.1.2 DWLRet

enum DWLRet

Enumerator

DWL_OK	the hardware is working correctly
DWL_ERROR	the hardware is running in error
DWL_HW_WAIT_OK	the hardware is working correctly
DWL_HW_WAIT_ERROR	the hardware is running in error
DWL_HW_WAIT_TIMEOUT	the hardware is time out

3.1.1.3 DWLClientType

enum DWLClientType

Enumerator

DWL_CLIENT_TYPE_H264_DEC	The client type is h264.

Enumerator

DWL_CLIENT_TYPE_MPEG4_DEC	The client type is mpeg4.
DWL_CLIENT_TYPE_JPEG_DEC	The client type is jpeg.
DWL_CLIENT_TYPE_PP	The client type is pp.
DWL_CLIENT_TYPE_VC1_DEC	The client type is vc1.
DWL_CLIENT_TYPE_MPEG2_DEC	The client type is mpeg2.
DWL_CLIENT_TYPE_VP6_DEC	The client type is vp6.
DWL_CLIENT_TYPE_AVS_DEC	The client type is avs.
DWL_CLIENT_TYPE_RV_DEC	The client type is rv.
DWL_CLIENT_TYPE_VP8_DEC	The client type is vp8.
DWL_CLIENT_TYPE_VP9_DEC	The client type is vp9.
DWL_CLIENT_TYPE_HEVC_DEC	The client type is hevc.
DWL_CLIENT_TYPE_ST_PP	The client type is pp.
DWL_CLIENT_TYPE_H264_MAIN10	The client type is h264_main10.
DWL_CLIENT_TYPE_AVS2_DEC	The client type is avs2.
DWL_CLIENT_TYPE_AV1_DEC	The client type is av1.
DWL_CLIENT_TYPE_VVC_DEC	The client type is vvc.
DWL_CLIENT_TYPE_MAX	The max client type number.

3.1.2 Functions

3.1.2.1 DWLReadAsicID()

```
u32 DWLReadAsicID (
            enum DWLClientType client_type )
```

Read the HW ID. This function does not need a DWL instance to run.

Parameters

in	*client_type*	Decoded stream type

Returns

u32 The HW ID

3.1.2.2 DWLReadCoreHwBuildID()

```
u32 DWLReadCoreHwBuildID (
            u32 core_id )
```

Read the HW build ID. This function does not need a DWL instance to run.

Parameters

in	*core_id*	The core ID
in	*client_type*	Decoded stream type

Returns

u32 The HW Build ID

3.1.2.3 DWLGetReleaseHwFeaturesByClientType()

```
u32 DWLGetReleaseHwFeaturesByClientType (
            enum DWLClientType client_type,
            const void ** p_hw_feature )
```

Get HW feature list by client type. This function does not need a DWL instance to run.

Parameters

in	*p_hw_feature*	pointer to hardware feature list.
in	*client_type*	Decoded stream type

Returns

u32 0 for success

3.1.2.4 DWLReadAsicCoreCount()

```
u32 DWLReadAsicCoreCount (
            void  )
```

Get the number of ASIC cores, which is the all subsys numbers.

Returns

u32 The number of ASIC cores.

3.1.2.5 DWLInit()

```
const void * DWLInit (
            struct DWLInitParam * param )
```

Initialize a DWL instance.

Parameters

in	*param*	Pointer to the DWLInitParam structure.

Returns

void* Pointer to a DWL instance.

3.1.2.6 DWLRelease()

```
enum DWLRet DWLRelease (
            const void * instance )
```

Release a DWl instance.

Parameters

in	*instance*	Pointer to the DWL instance to be released.

Returns

DWLRet DWL_OK, DWL_ERROR

3.1.2.7 DWLReserveHw()

```
enum DWLRet DWLReserveHw (
            const void * instance,
            struct DWLReqInfo * info,
            i32 * core_id )
```

Reserves the hardware resource for one codec instance.
This function is part of the hardware sharing mechanism in use for multi-instance port-processor support. When called it shall block and return when exclusive access to the required hardware can be granted to the calling client.

Parameters

in	*instance*	Pointer to a DWL instance.
in	*core_id*	Pointer to core_id.
in	*client_type*	Decoded stream type.

Returns

DWLRet DWL_OK, DWL_ERROR

3.1.2.8 DWLReleaseHw()

```
enum DWLRet DWLReleaseHw (
            const void * instance,
            i32 core_id )
```

Releases a previously reserved hardware resource. This function is called when the hardware has finished decoding frames.

Parameters

in	*instance*	Pointer to a DWL instance.
in	*core_id*	Pointer to core_id.

Returns

DWLRet DWL_OK.

3.1.2.9 DWLMallocRefFrm()

```
enum DWLRet DWLMallocRefFrm (
            const void * instance,
            u64 size,
            struct DWLLinearMem * info )
```

Allocates a contiguous linear block of memory for the reference frame buffer. The buffer has to be a contiguous, linear memory buffer residing in the physical memory.

Parameters

in	*instance*	Pointer to a DWL instance.
in	*size*	Size of the requested memory buffer (in bytes). Upon return, this parameter contains the exact size of the allocated memory area.
in	*info*	Pointer to the DWLLinearMem structure for the returned memory block.

Returns

DWLRet DWL_OK, DWL_ERROR

3.1.2.10 DWLFreeRefFrm()

```
void DWLFreeRefFrm (
            const void * instance,
            struct DWLLinearMem * info )
```

Frees a block of memory for the reference frame buffer previously allocated with DWLMallocRefFrm.

Parameters

in	*instance*	Pointer to a DWL instance.
in	*info*	Pointer to the DWLLinearMem structure for the returned memory block.

Returns

void None

3.1.2.11 DWLMallocLinear()

```
enum DWLRet DWLMallocLinear (
            const void * instance,
```

```
        u64 size,
        struct DWLLinearMem * info )
```

Allocates a contiguous, linear block of memory for the software/hardware shared memory buffer. The buffer has to be a contiguous, linear memory buffer residing in the physical memory.

Parameters

in	*instance*	Pointer to a DWL instance.
in	*size*	Size of the requested memory buffer (in bytes).
in	*info*	Pointer to the DWLLinearMem structure for the memory block allocated.

Returns

DWLRet DWL_OK, DWL_ERROR

3.1.2.12 DWLFreeLinear()

```
void DWLFreeLinear (
        const void * instance,
        struct DWLLinearMem * info )
```

Frees a block of memory for the software/hardware shared memory previously allocated with DWLMallocLinear.

Parameters

in	*instance*	Pointer to a DWL instance.
in	*info*	Pointer to the DWLLinearMem structure for the memory block to free.

Returns

void None

3.1.2.13 DWLWriteReg()

```
void DWLWriteReg (
            const void * instance,
            i32 core_id,
            u32 offset,
            u32 value )
```

Writes a 32-bit value to a specified hardware register. All registers are written at once at the beginning of frame decoding.

Parameters

in	*instance*	Pointer to a DWL instance.
in	*core_id*	Core ID of the hardware resource to release.
in	*offset*	Register offset is relative to the hardware ID register (#0) in byte.
in	*value*	Value to write to the hardware register.

Returns

void None

3.1.2.14 DWLWriteRegs()

```
void DWLWriteRegs (
            const void * instance,
            i32 core_id,
            u32 offset,
            u32 * from,
            u32 num_regs,
            u32 print_flag )
```

Writes num 32-bit value to a specified hardware register. All registers are written at once at the beginning of frame decoding.

Parameters

in	*instance*	Pointer to a DWL instance.
in	*core_id*	Core ID of the hardware resource to release.
in	*offset*	Register offset is relative to the hardware ID register (#0) in byte.
in	*from*	Source Register address.
in	*num_regs*	The number of Registers should be Written.
in	*print_flag*	Whether print regs info.

3.1.2.15 DWLReadReg()

```
u32 DWLReadReg (
            const void * instance,
            i32 core_id,
            u32 offset )
```

Reads and returns a 32-bit value from a specified hardware register. The status register is read after every macroblock decoding by software.

Parameters

in	*instance*	Pointer to a DWL instance.
in	*core_id*	Core ID of the hardware resource to release.
in	*offset*	Register offset is relative to the hardware ID register (#0) in byte.

Returns

u32 The value of the specified hardware register.

3.1.2.16 DWLEnableHw()

```
void DWLEnableHw (
            const void * instance,
```

```
        i32 core_id,
        u32 offset,
        u32 value )
```

Enables the hardware.

Parameters

in	*instance*	Pointer to a DWL instance.
in	*core_id*	Core ID of the hardware resource to release.
in	*offset*	Register offset.
in	*value*	Value to enable hardware.

Returns

void None

3.1.2.17 DWLDisableHw()

```
void DWLDisableHw (
        const void * instance,
        i32 core_id,
        u32 offset,
        u32 value )
```

This interface is a particular register writing function that stops and disables the hardware by writing a specific register.

Parameters

in	*instance*	Pointer to a DWL instance
in	*core_id*	Core ID of the hardware resource to release.
in	*offset*	Register offset.
in	*value*	Value to disable hardware.

Returns

void None

3.1.2.18 DWLWriteRegToHw()

```
void DWLWriteRegToHw (
            const void * instance,
            i32 core_id,
            u32 offset,
            u32 value )
```

Writes a 32-bit value to a specified hardware register. This register is written during the frame decoding.

Parameters

in	*instance*	Pointer to a DWL instance.
in	*core_id*	Core ID of the hardware resource to release.
in	*offset*	Register offset is relative to the hardware ID register (#0) in bytes.
in	*value*	Value to write to the hardware register.

Returns

void None

3.1.2.19 DWLReadRegFromHw()

```
u32 DWLReadRegFromHw (
            const void * instance,
            i32 core_id,
            u32 offset )
```

Reads and returns a 32-bit value from a specified hardware register when hardware is running.

Parameters

in	*instance*	Pointer to a DWL instance.
in	*core_id*	Core ID of the hardware resource to release.
in	*offset*	Register offset is relative to the hardware ID register (#0) in bytes.

Returns

u32 The value of the specified hardware register.

3.1.2.20 DWLWaitHwReady()

```
enum DWLRet DWLWaitHwReady (
            const void * instance,
            i32 core_id,
            u32 timeout )
```

Synchronizes the software with the hardware.

Parameters

in	*instance*	Pointer to a DWL instance.
in	*core_id*	Core ID of the hardware resource to release.
in	*timeout*	Timeout period in millisecond. 0: do not wait. -1: infinite wait.

Returns

DWLRet DWL_HW_WAIT_OK, DWL_HW_WAIT_ERROR, DWL_HW_WAIT_TIMEOUT

3.1.2.21 DWLSetIRQCallback()

```
void DWLSetIRQCallback (
            const void * instance,
```

```
        i32 core_id,
        DWLIRQCallbackFn * callback_fn,
        void * arg )
```

Sets the IRQ callback function.

Parameters

in	*instance*	Pointer to a DWL instance.
in	*core_id*	Core ID of the hardware resource to release.
in	*callback_fn*	Pointer to the callback function.
in	*arg*	Arguments set for callback function.

Returns

void None

3.1.2.22 DWLReadPpConfigure()

```
void DWLReadPpConfigure (
        const void * instance,
        u32 core_id,
        void * ppu_cfg,
        u32 pjpeg_coeff_buffer_size )
```

Read the pp configuration.

Parameters

in	*instance*	Pointer to a DWL instance.
in	*core_id*	Core ID of the hardware resource to release.
in	*ppu_cfg*	Pointer to the PpUnitIntConfig stucture.
in	*pjpeg_coeff_buffer_size*	The pjpeg coeff buffer size.(Pjpeg only).

Returns

void None

3.1.2.23 DWLReadMcRefBuffer()

```
void DWLReadMcRefBuffer (
            const void * instance,
            u32 core_id,
            u64 buf_size,
            u32 dpb_size )
```

DWLReadMcRefBuffer (Not support).

Parameters

in	*instance*	Pointer to a DWL instance.
in	*core_id*	The core id.
in	*buf_size*	Buffer bytes allocated.
in	*dpb_size*	Decoded picture buffer size.

Returns

void None

3.1.2.24 DWLmalloc()

```
void * DWLmalloc (
            size_t n )
```

Allocates a block of n bytes of memory, returning a pointer to the beginning of the block.

Parameters

in	*n*	Size of memory block in bytes

Returns

void* On success, a pointer to the memory block allocated by the function. If the function failed to allocate the requested block of memory, a NULL pointer is returned.

3.1.2.25 DWLfree()

```
void DWLfree (
            void * p )
```

Deallocates a block of memory, previously allocated by a call to DWLmalloc() or DWLcalloc().

Parameters

in	*p*	Pointer to a memory block previously allocated with DWLmalloc() or DWLcalloc().

Returns

void None

3.1.2.26 DWLcalloc()

```
void * DWLcalloc (
            size_t n,
            size_t s )
```

Allocates a block of memory for an array of n elements, each of them s bytes long, and initializes all its bits to zero.

Parameters

in	*n*	Number of elements to allocate.
in	*s*	Size of each element.

Returns

void * On success, a pointer to the memory block is allocated by the function. If the function failed to allocate the requested block of memory, a NULL pointer is returned.

3.1.2.27 DWLmemcpy()

```
void * DWLmemcpy (
            void * d,
            const void * s,
            size_t n )
```

Copies the values of n bytes from the location pointed to by s directly to the memory block pointed to by d.

Parameters

in	*d*	Pointer to the destination array where the content is to be copied.
in	*s*	Pointer to the source of data to be copied.
in	*n*	Number of bytes to copy.

Returns

void * A pointer to the destination array, d, is returned.

3.1.2.28 DWLmemset()

```
void * DWLmemset (
            void * d,
```

```
            i32 c,
            size_t n )
```

Allocates a block of memory for an array of n elements, each of them s bytes long, and initializes all its bits to zero.

Parameters

in	*d*	Pointer to the block of memory to fill.
in	*c*	Value to be set.
in	*n*	Number of bytes to be set to the value c.

Returns

void ∗ A pointer to the block of memory to fill, d, is returned.

3.1.2.29 DWLVcmdIsUsed()

```
u32 DWLVcmdIsUsed (
            void  )
```

Indicates if vcmd used.

Parameters

in	*void*	None.

Returns

u32 The num of subsystems with vcmd.

3.1.2.30 DWLReserveCmdBuf()

```
enum DWLRet DWLReserveCmdBuf (
            const void * instance,
```

```
            struct DWLReqInfo * info,
            u32 * cmd_buf_id )
```

Reserve one valid command buffer.

Parameters

in	*instance*	Pointer to a DWL instance.
in	*client_type*	Decoded stream type.
in	*width*	The video decoding width.
in	*height*	The video decoding height.
in	*cmd_buf_id*	A pointer to the cmd buf id

Returns

DWLRet DWL_OK, DWL_ERROR

3.1.2.31 DWLEnableCmdBuf()

```
enum DWLRet DWLEnableCmdBuf (
            const void * instance,
            u32 cmd_buf_id )
```

Enable one command buffer: link and enable.

Parameters

in	*instance*	Pointer to a DWL instance.
in	*cmd_buf_id*	A pointer to the cmd buf id.

Returns

DWLRet DWL_OK, DWL_ERROR

3.1.2.32 DWLWaitCmdBufReady()

```
enum DWLRet DWLWaitCmdBufReady (
            const void * instance,
            u16 cmd_buf_id )
```

Wait cmd buffer ready.

Parameters

in	*instance*	Pointer to a DWL instance.
in	*cmd_buf_id*	A pointer to the cmd buf id.

Returns

DWLRet DWL_OK, DWL_ERROR

3.1.2.33 DWLReleaseCmdBuf()

```
enum DWLRet DWLReleaseCmdBuf (
            const void * instance,
            u32 cmd_buf_id )
```

Release one command buffer.

Parameters

in	*instance*	Pointer to a DWL instance.
in	*cmd_buf_id*	A pointer to the cmd buf id.

Returns

DWLRet DWL_OK, DWL_ERROR

3.1.2.34 DWLWaitCmdbufsDone()

```
enum DWLRet DWLWaitCmdbufsDone (
            const void * instance,
            const void * owner )
```

Wait all left cmd bufs done.

Parameters

in	*instance*	Pointer to a DWL instance.
in	*owner*	A pointer to decoder instance.

Returns

DWLRet DWL_OK, DWL_ERROR

3.1.2.35 DWLFlushRegister()

```
enum DWLRet DWLFlushRegister (
            const void * instance,
            u32 cmd_buf_id,
            u32 * dec_regs,
            u32 * mc_fresh_regs,
            u32 mc_buf_id )
```

Flush vcmd register.

Parameters

in	*instance*	Pointer to a DWL instance.
in	*cmd_buf_id*	A pointer to a decoder instance.
in	*dec_regs*	A pointer to decoder regs.
in	*mc_fresh_regs*	A pointer to mc fresh regs based the current core
in	*mc_buf_id*	The mc buf id.
in	*owner*	A pointer to a decoder instance.

Returns

DWLRet DWL_OK, DWL_ERROR

3.1.2.36 DWLRefreshRegister()

```
enum DWLRet DWLRefreshRegister (
            const void * instance,
            u32 cmd_buf_id,
            u32 * dec_regs )
```

Refresh register, nothing needs to do, already refresh in DWLWaitCmdBufReady().

Parameters

in	*instance*	Pointer to a DWL instance.
in	*cmd_buf_id*	The cmd buf id.
in	*dec_regs*	The pointer to decoder regs.

Returns

DWLRet DWL_OK, DWL_ERROR

3.1.2.37 DWLVcmdMCRefreshStatusRegs()

```
enum DWLRet DWLVcmdMCRefreshStatusRegs (
            const void * instance,
            u32 * dec_regs,
            u32 cmdbuf_id )
```

Refresh the status regs of requirement for Vcmd multicore.

Parameters

in	*instance*	Pointer to a DWL instance.
in	*dec_regs*	The pointer to decoder regs.
in	*cmdbuf_id*	The cmd buf id.

Returns

DWLRet DWL_OK, DWL_ERROR

3.1.2.38 DWLGetVcmdMCVirtualCoreId()

```
i32 DWLGetVcmdMCVirtualCoreId (
            const void * instance,
            u32 cmdbuf_id )
```

Parameters

in	*instance*	Pointer to a DWL instance.
in	*cmdbuf_id*	The cmd buf id.

Returns

i32 The current core id.

3.1.2.39 DWLCmdPushSliceRegs()

```
void DWLCmdPushSliceRegs (
            const void * instance,
            u32 cmd_buf_id,
            u32 * regs )
```

Parameters

in	*instance*	Pointer to a DWL instance.
in	*cmdbuf_id*	The cmd buf id.
in	*regs*	regs need to flush.

Returns

void None

3.1.2.40 DWLReadByte()

```
u8 DWLReadByte (
            const u8 * p )
```

Read one byte from a given pointer.

Parameters

in	*p*	Pointer to the memory block to read.

Returns

u8 One byte data from the given address.

3.1.2.41 DWLPrivateAreaReadByte()

```
u8 DWLPrivateAreaReadByte (
            const u8 * p )
```

Read one byte from the memory block pointed to by p.
This function is just an interface to access private or protected buffer; it should be realized by customer.

Parameters

in	*p*	Pointer to a virtual address in the private buffer.

Returns

u8 One byte data from the given address.

3.1.2.42 DWLPrivateAreaWriteByte()

```
void DWLPrivateAreaWriteByte (
            u8 * p,
            u8 data )
```

Write one byte data to the memory block pointed to by p.
This function is just an interface to access private or protected buffer; it should be realized by customer.

Parameters

in	*p*	Pointer to the memory block to write.
in	*data*	One byte of data to write.

Returns

void None

3.1.2.43 DWLPrivateAreaMemcpy()

```
void * DWLPrivateAreaMemcpy (
            void * d,
            const void * s,
            u32 n )
```

Copies the values of n bytes from the location pointed to by s directly to the memory block pointed to by d.
This function is just an interface to access private or protected buffer; it should be realized by customer.

Parameters

in	*d*	Pointer to the destination array where the content is to be copied.
in	*s*	Pointer to the source of data to be copied.
in	*n*	Number of bytes to copy.

Returns

void * A pointer to the destination array, d, is returned.

3.1.2.44 DWLPrivateAreaMemset()

```
void * DWLPrivateAreaMemset (
            void * p,
            i32 c,
            u32 n )
```

Sets the first n bytes of the block of memory pointed to by d to the specified value c.
This function is just an interface to access private or protected buffer; it should be realized by customer.

Parameters

in	*p*	Pointer to the block of memory to fill.
in	*c*	Value to be set.
in	*n*	number of bytes to be set to the value c.

Returns

void * A pointer to the block of memory to fill, d, is returned.

3.1.2.45 DWLDMATransData()

```
void DWLDMATransData (
            addr_t device_bus_addr,
            void * host_virtual_addr,
            u32 length,
            enum DWLDMADirection dir )
```

DMA data transfer between host memory and device memory.

Parameters

in	*device_bus_addr*	Device memory address
in	*host_virtual_addr*	Host memory address
in	*length*	Copy data length(byte)
in	*dir*	DMA transfer direction. -HOST_TO_DEVICE. -DEVICE_TO_HOST.

Returns

void None

4 Data Structure Documentation

4.1 DWL

Decoder wrapper layer functionality.

Data Fields

- enum DWLRet(∗ **ReserveHw**)(const void ∗instance, struct DWLReqInfo ∗info, i32 ∗core_id)
  reserve hw
- enum DWLRet(∗ **ReleaseHw**)(const void ∗instance, i32 core_id)
  release hw
- enum DWLRet(∗ **MallocLinear**)(const void ∗instance, u64 size, struct DWLLinearMem ∗info)
  malloc physical, linear memory
- void(∗ **FreeLinear**)(const void ∗instance, struct DWLLinearMem ∗info)
  freee physical, linear memory
- void(∗ **WriteReg**)(const void ∗instance, i32 core_id, u32 offset, u32 value)
  Register access, write regs.
- u32(∗ **ReadReg**)(const void ∗instance, i32 core_id, u32 offset)
  Register access, write regs.
- void(∗ **EnableHw**)(const void ∗instance, i32 core_id, u32 offset, u32 value)
  HW starting.
- void(∗ **DisableHw**)(const void ∗instance, i32 core_id, u32 offset, u32 value)
  HW starting.
- enum DWLRet(∗ **WaitHwReady**)(const void ∗instance, i32 core_id, u32 timeout)
  HW synchronization.
- void(∗ **SetIRQCallback**)(const void ∗instance, i32 core_id, DWLIRQCallbackFn ∗callback_fn, void ∗arg)

HW synchronization callback function, used for multi core.

- void ∗(∗ **Malloc**)(size_t n)

  malloc virtual memory .

- void(∗ **Free**)(void ∗p)

  fress virtual memory .

- void ∗(∗ **Calloc**)(size_t n, size_t s)

  malloc virtual memory .

- void ∗(∗ **Memcpy**)(void ∗d, const void ∗s, size_t n)

  memory copy for virtual memory .

- void ∗(∗ **Memset**)(void ∗d, i32 c, size_t n)

  memset virtual memory .

- i32(∗ **Pthread_create**)(pthread_t ∗tid, const pthread_attr_t ∗attr, void ∗(∗start)(void ∗), void ∗arg)

  POSIX compatible threading functions.

- void(∗ **Pthread_exit**)(void ∗value_ptr)

  POSIX compatible threading functions.

- i32(∗ **Pthread_join**)(pthread_t thread, void ∗∗value_ptr)

  POSIX compatible threading functions.

- i32(∗ **Pthread_mutex_init**)(pthread_mutex_t ∗mutex, const pthread_mutexattr_t ∗attr)

  POSIX compatible threading functions.

- i32(∗ **Pthread_mutex_destroy**)(pthread_mutex_t ∗mutex)

  POSIX compatible threading functions.

- i32(∗ **Pthread_mutex_lock**)(pthread_mutex_t ∗mutex)

  POSIX compatible threading functions.

- i32(∗ **Pthread_mutex_unlock**)(pthread_mutex_t ∗mutex)

  POSIX compatible threading functions.

- i32(∗ **Pthread_cond_init**)(pthread_cond_t ∗cond, const pthread_condattr_t ∗attr)

  POSIX compatible threading functions.

- i32(∗ **Pthread_cond_destroy**)(pthread_cond_t ∗cond)

  POSIX compatible threading functions.

- i32(∗ **Pthread_cond_wait**)(pthread_cond_t ∗cond, pthread_mutex_t ∗mutex)

  POSIX compatible threading functions.

- i32(∗ **Pthread_cond_signal**)(pthread_cond_t ∗cond)

  POSIX compatible threading functions.

- int(∗ **Printf**)(const char ∗string,...)

  API trace function. Set to NULL if no trace wanted.

4.2 DWLCodecConfig

Used for IP-integration codec config.DWLCodecConfig is used for IP-integration codec config.

Data Fields

- u32 **input_yuv_swap**

  Set swaps for encoder frame input data. From MSB to LSB: swap each 128 bits, 64, 32, 16, 8.
- u32 **output_swap**

  Set swaps for encoder frame output (stream) data. From MSB to LSB: swap each 128 bits, 64, 32, 16, 8.
- u32 **prob_table_swap**

  Set swaps for encoder probability tables. From MSB to LSB: swap each 128 bits, 64, 32, 16, 8.
- u32 **ctx_counter_swap**

  Set swaps for encoder context counters. From MSB to LSB: swap each 128 bits, 64, 32, 16, 8.
- u32 **statistics_swap**

  Set swaps for encoder statistics. From MSB to LSB: swap each 128 bits, 64, 32, 16, 8.
- u32 **fgbg_map_swap**

  Set swaps for encoder foreground/background map. From MSB to LSB: swap each 128 bits, 64, 32, 16, 8.
- bool **irq_enable**

  ASIC interrupt enable. This enables/disables the ASIC to generate interrupt.
- u32 **axi_read_id_base**

  AXI master read ID base. Read service type specific values are added to this.
- u32 **axi_write_id_base**

  AXI master write id base. Write service type specific values are added to this.
- u32 **axi_read_max_burst**

  AXI maximum read burst issued by the core [1..511].
- u32 **axi_write_max_burst**

  AXI maximum write burst issued by the core [1..511].
- bool **clock_gating_enabled**

  ASIC clock gating enable. This enables/disables the ASIC to utilize clock gating. If this is 'true', ASIC core clock is automatically shut down when the encoder enable bit is '0' (between frames).
- u32 **timeout_period**

  ASIC internal timeout period in clocks. Timeout counter acts as a watchdog that asserts a timeout interrupt if bus is idle for the set period while the encoder is enabled. Set to zero to disable.

4.3 DWLHwFuseStatus

A structure containing HW fuse configuration information.

Data Fields

- u32 **vp6_support_fuse**

  HW supports VP6.
- u32 **vp7_support_fuse**

  HW supports VP7.
- u32 **vp8_support_fuse**

  HW supports VP8.
- u32 **vp9_support_fuse**

  HW supports VP9.
- u32 **h264_support_fuse**

  HW supports H.264.
- u32 **HevcSupportFuse**

  HW supports HEVC.
- u32 **mpeg4_support_fuse**

  HW supports MPEG-4.
- u32 **mpeg2_support_fuse**

  HW supports MPEG-2.
- u32 **sorenson_spark_support_fuse**

  HW supports Sorenson Spark.
- u32 **jpeg_support_fuse**

  HW supports JPEG.
- u32 **vc1_support_fuse**

  HW supports VC-1 Simple.
- u32 **pp_standalone_fuse**

  HW supports post-processor.
- u32 **pp_config_fuse**

  HW post-processor functions bitmask.
- u32 **max_dec_pic_width_fuse**

  Maximum video decoding width supported
- u32 **max_pp_out_pic_width_fuse**

  Maximum output width of Post-Processor.
- u32 **avs_support_fuse**

  HW supports AVS.
- u32 **rv_support_fuse**

  HW supports RealVideo codec.
- u32 **custom_mpeg4_support_fuse**

  Fuse for custom MPEG-4.

4.4 DWLInitParam

A structure used to pass parameters during initialization of the decoder wrapper layer.

Data Fields

- enum DWLClientType **client_type**

  Decoded stream type.
- char * **dec_dev**

  hantrodec select
- char * **mem_dev**

  memalloc select
- void * **priv**

  private extension interface

4.5 DWLLinearMem

Linear memory area descriptor.

Data Fields

- u32 * **virtual_address**

  Pointer to the virtual address of stream linear buffer.
- addr_t **bus_address**

  Bus address of the linear buffer. The start of the stream buffer must be in a 128-bit aligned position or accessible for reading from the previous 128-bit aligned position.
- u64 **size**

  physical size (rounded to page multiple)
- u64 **logical_size**

  requested size in bytes
- u32 **mem_type**

  type for secure mode
- void * **priv**

  private extension interface

4.6 DWLReqInfo

DWLReqInfo is used for DWLReserveHw and DWLReserveCmdBuf.Send the VCD source information.

Data Fields

- u32 **core_mask**

  core_mask for user to select cores: [0,15]core mask, [16,31]client type
- u32 **width**

  Picture width.
- u32 **height**

  Picture height.
- void ∗ **owner**

  instance ctx

4.7 strmInfo

low latency mode descriptor.

Data Fields

- u32 **low_latency**

  Flag that if stream run in low latecny mode.
- u32 **send_len**

  Data len perpare for hw.
- addr_t **strm_bus_addr**

  Host physical address decode node for current stream in buffer.
- addr_t **strm_bus_start_addr**

  Host physical start address for current buffer.
- u8 ∗ **strm_vir_addr**

  Virtual address decode node for current stream in buffer.
- u8 ∗ **strm_vir_start_addr**

  Virtual start address for current buffer.
- u32 **last_flag**

Flag that last data for cureent stream.

- addr_t **strm_buff**

  input stream buffer virtual address.

- addr_t **buff_size**

  input stream buffer size.

- addr_t **strm_buff_bus_address**

  input stream buffer bus address.

Document Revision History

This section describes top level differences in the versions of this document.

Note

This document is not necessarily updated for each patch or minor revision. The information in this document tends to be stable across a revision (nnn) series.

Document Revision	Date	Compatible Core	Description
1.03	2024-03-28	VC9000D	Reconstructed and refined the contents of *Overview* and *Software Integration*.
1.02	2024-01-02	VC9000D	Updated the document name and template.
1.01	2022-11-30	VC9000D	Initial release.